6.1 Problem description

The company Steel has received an order for 500 tonnes of steel to be used in shipbuilding. This steel must have the following characteristics ('grades').

Table 6.1: Characteristics of steel ordered

Chemical element	Minimum grade	Maximum grade
Carbon (C)	2	3
Copper (Cu)	0.4	0.6
Manganese (Mn)	1.2	1.65

The company has seven different raw materials in stock that may be used for the production of this steel. Table 6.2 lists the grades, available amounts and prices for all raw materials.

Table 6.2: Raw material grades, availabilities, and prices

Raw material	C %	Cu %	Mn %	Availability in t	Cost in €/t
Iron alloy 1	2.5	0	1.3	400	200
Iron alloy 2	3	0	0.8	300	250
Iron alloy 3	0	0.3	0	600	150
Copper alloy 1	0	90	0	500	220
Copper alloy 2	0	96	4	200	240
Aluminum alloy 1	0	0.4	1.2	300	200
Aluminum alloy 2	0	0.6	0	250	165

The objective is to determine the composition of the steel that minimizes the production cost.

6.1.1 Model formulation

We use *RAW* to represent the set of raw materials and *COMP* for the set of components (chemical elements) that are relevant for the grade requirements. We want to determine the quantity use_r of every

raw material r that is used to produce the quantity *produce* of steel to satisfy the given demand *DEM*. We denote by P_{rc} the percentage of the chemical element c in raw material r and $COST_r$ the buying price per kg of r. Furthermore, the minimum and maximum grades $PMIN_c$ and $PMAX_c$ are given for every component c. We obtain the following mathematical model:

$$\text{minimize} \sum_{r \in RAW} COST_r \cdot use_r \tag{6.1.1}$$

$$produce = \sum_{r \in RAW} use_r \tag{6.1.2}$$

$$\forall c \in COMP : \sum_{r \in RAW} P_{rc} \cdot use_r \geq PMIN_c \cdot produce \tag{6.1.3}$$

$$\forall c \in COMP : \sum_{r \in RAW} P_{rc} \cdot use_r \leq PMAX_C \cdot produce \tag{6.1.4}$$

$$\forall r \in RAW : use_r \leq AVAIL_r \tag{6.1.5}$$

$$produce \geq DEM \tag{6.1.6}$$

$$\forall r \in RAW : use_r \geq 0, produce \geq 0 \tag{6.1.7}$$

The objective, given by (6.1.1), is to minimize the total cost that is calculated as the sum of the raw material prices times the quantity used. The constraint (6.1.2) indicates that the resulting product weight is equal to the sum of the weights of the raw materials used for its production.
The constraints (6.1.3) and (6.1.4) impose the limits on the grade of the final product. They have been obtained based on the observation that if the steel contains use_r tonnes of a raw material that contains P_{rc} percent of a chemical element c, then the quantity of this element in the steel is given by the sum

$$\sum_{r \in RAW} P_{rc} \cdot use_{rf}$$

and the percentage of the element c in the final product is given by the ratio

$$\frac{\sum_{r \in RAW} P_{rc} \cdot use_r}{produce}$$

Adding the lower bound $PMIN_c$ to express the constraint on the minimum grade of c results in the following relation:

$$\frac{\sum_{r \in RAW} P_{rc} \cdot use_{rf}}{produce} \geq PMIN_c$$

This relation is non-linear but it can be transformed by multiplying it with *produce* – and we finally obtain the linear constraint (6.1.3). The constraint (6.1.4) is derived following the same scheme.

The constraints (6.1.5) make sure that only the available quantities of raw material are used. The constraint (6.1.6) guarantees that the produced amount of steel satisfies the demand. The last set of constraints (6.1.7) establishes the non-negativity condition for all variables of the problem.

6.1.2 Code

```
#include <ilcplex/ilocplex.h>
#include <fstream>
#include <vector>
#include"cmath"
#include"conio.h"
```

```
ILOSTLBEGIN
typedef IloArray<IloNumArray>        IloNumArray2base;
typedef IloArray<IloNumVarArray>     IloNumVarArray2;

IloInt i, j, P, Q, RAW, COMP;
IloNumArray Cost, Avail, PMin, PMax;
IloNumArray2 C;
IloNum DEM;

int main()

{

                      ofstream fout("SolTP-1.xls");

            // creating an environment
            IloEnv env;
            try {

                  //define_data(env);

                  Cost = IloNumArray(env);
                  Avail = IloNumArray(env);

                  C = IloNumArray2(env);

                  PMin = IloNumArray(env);
                  PMax = IloNumArray(env);


                ifstream fin("data1.txt");


                  fin >> Cost >> Avail >> C >> PMin >> PMax >> DEM;

                  if (!fin)

                  {
                        cout << "Unable to open";
                        exit(9);
                  }


                  RAW = Cost.getSize();
```

```
COMP = PMin.getSize();

std::vector<double> xx(RAW);

IloModel model(env);


// variable declaration
/*----------------------------*/


IloNumVarArray x(env, RAW, 0, IloInfinity, ILOFLOAT);
IloNumVar y(env, 0, IloInfinity, ILOFLOAT);


//variable declaration ends

/*OBJECTIVE FUNCTION*/

//add the objective to the model

model.add(IloMinimize(env, IloScalProd(Cost, x)));

//adding the constraints to the model


// Constraint 1
{
      model.add(IloSum(x) == y);
}

//Constraint 2

for (i = 0; i < COMP; i++)

{
      IloExpr DD(env);
      for (j = 0; j < RAW; j++)
      {
            DD += C[i][j] * x[j];
      }
      model.add(DD >= PMin[i] * y);
}

//Constraint 3

for (i = 0; i < COMP; i++)
```

```
                {
                      IloExpr DD1(env);
                      for (j = 0; j < RAW; j++){
                            DD1 += C[i][j] * x[j];
                      }
                      model.add(DD1 <= PMax[i] * y);
                }

                // Constraint 4
                for (j = 0; j < RAW; j++)
                {
                      model.add(x[j] <= Avail[j]);
                }
                // Constraint 5
                model.add(y >= DEM);

                IloCplex cplex(model);
                cplex.extract(model);
                cplex.solve();

                // outcome of the solver.
                env.out() << "Solution Status is " << cplex.getStatus()
<< endl;
                fout << "Solution Status is " << cplex.getStatus() <<
endl;
                // the results
                env.out() << " The objective value is " <<
cplex.getObjValue() << endl;
                fout << " The objective value is " << "\t" <<
cplex.getObjValue() << endl;

                cplex.exportModel("model.lp");

                for (int i = 0; i < RAW; ++i) {
                      xx[i] = cplex.getValue(x[i]);
                      fout << "xx[" << i << "]\t" << xx[i] << '\n';
                }


            }
            catch (IloException& ex)
            {
                  cerr << "Error: " << ex << endl;
            }
            catch (...)
            {
```

```
                cerr << "Error" << endl;
            }
            // ending the program
            env.end();

      return 0;

}


/* Solution
Solution Status is Optimal
 The objective value is      98121.6
 xx[0]       400
 xx[1]       0
 xx[2]       39.7763
 xx[3]       0
 xx[4]       2.76127
 xx[5]       57.4624
 xx[6]       0
*/
```

6.1.3 Data

```
[200,250,150,220,240,200,165]
[400,300,600,500,200,300,250]
[[2.5,3,0,0,0,0,0],[0,0,0.3,90,96,0.4,0.6],[1.3,0.8,0,0,4,1.2,0]]
[2,0.4,1.2]
[3,0.6,1.65]
500
```

6.2 Problem description

The company CowFood produces food for farm animals that is sold in two forms: powder and granules. The raw materials used for the production of the food are: oat, maize and molasses. The raw materials (with the exception of molasses) first need to be ground, and then all raw materials that will form a product are blended. In the last step of the production process the product mix is either transformed to granules or sieved to obtain food in the form of powder.

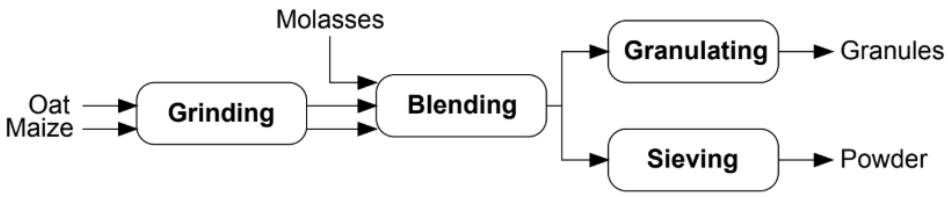

Figure 6.1: Animal food production process

Every food product needs to fulfill certain nutritional requirements. The percentages of proteins, lipids and fibers contained in the raw materials and the required percentages in the final products are listed in Table 6.3.

Table 6.3: Contents of nutritional components in percent

Raw material	Proteins	Lipids	Fiber
Oat	13.6	7.1	7
Maize	4.1	2.4	3.7
Molasses	5	0.3	25
Required contents	≥ 9.5	≥ 2	≤ 6

There are limits on the availability of raw materials. Table 6.4 displays the amount of raw material that is available every day and the respective prices.

Table 6.4: Raw material availabilities and prices

Raw material	Available amount in kg	Cost in €/kg
Oat	11900	0.13
Maize	23500	0.17
Molasses	750	0.12

The cost of the different production steps are given in the following table.

Table 6.5: Production costs in €/kg

Grinding	Blending	Granulating	Sieving
0.25	0.05	0.42	0.17

With a daily demand of nine tonnes of granules and twelve tonnes of powder, which quantities of raw materials are required and how should they be blended to minimize the total cost?

6.2.1 Model formulation

This model is very similar to the preceding except that we now want to produce **two** products and that instead of the minimum and maximum grades we have to fulfill certain nutritional requirements. Let $FOOD = \{1, 2\}$ (1 = granules, 2 = powder) be the set of food product types that are produced, RAW the set of raw materials, and $COMP = \{1, 2, 3\}$ the set of nutritional components (with 1 = protein, 2 = lipids, 3 = fiber). We further use $COST_r$ to denote the price per kg of raw material r, $PCOST_p$ for the cost of production process p in €/kg, REQ_c for the required content of nutritional component c, P_{rc} for the content of c in raw material r, $AVAIL_r$ for the maximum available quantity of raw material r, and DEM_f for the daily demand of food product f.
With variable use_{rf} representing the quantity of raw material r used for the production of food type f and $produce_f$ the amount of food f produced, we obtain the following model:

$$\begin{aligned}
\text{minimize} \sum_{r \in RAW} \sum_{f \in FOOD} COST_r \cdot use_{rf} &+ \sum_{r \in RAW} \sum_{\substack{f \in FOOD \\ r \neq molasses}} PCOST_{grinding} \cdot use_{rf} \\
&+ \sum_{r \in RAW} \sum_{f \in FOOD} PCOST_{blending} \cdot use_{rf} + \sum_{r \in RAW} PCOST_{granulating} \cdot use_{r1} \\
&+ \sum_{r \in RAW} PCOST_{sieving} \cdot use_{r2} && (6.2.1)
\end{aligned}$$

$$\forall f \in FOOD : \sum_{r \in RAW} use_{rf} = produce_f \tag{6.2.2}$$

$$\forall f \in FOOD, c \in 1..2 : \sum_{r \in RAW} P_{rc} \cdot use_{rf} \geq REQ_c \cdot produce_f \tag{6.2.3}$$

$$\forall f \in FOOD : \sum_{r \in RAW} P_{r3} \cdot use_{rf} \leq REQ_3 \cdot produce_f \tag{6.2.4}$$

$$\forall r \in RAW : \sum_{f \in FOOD} use_{rf} \leq AVAIL_r \tag{6.2.5}$$

$$\forall f \in FOOD : produce_f \geq DEM_f \tag{6.2.6}$$

$$\forall r \in RAW, f \in FOOD : use_{rf} \geq 0, \ \forall f \in FOOD : produce_f \geq 0 \tag{6.2.7}$$

The objective function (6.2.1) that is to be minimized is the sum of all costs. The first term is the price paid for the raw materials, the following terms correspond to the production costs of the different process steps. Grinding is applied to all raw materials except molasses, blending to all raw material. Granules ($f = 1$) are obtained through granulating and powder ($f = 2$) by sieving the blended product. The constraint (6.2.2) expresses the fact that the produced quantity of every food type corresponds to the sum of the raw material used for its production. Through the constraints (6.2.3) and (6.2.4) we obtain the required content of nutritional components. They are based on the same reasoning that we saw for the 'grades' in the previous example. The percentage for the content of a nutritional component c in a food product f is given by the following ratio:

$$\frac{\sum_{r \in RAW} P_{rc} \cdot use_{rf}}{produce_f} \tag{6.2.8}$$

This percentage must be at least REQ_c for the first two components (protein, lipid) and at most REQ_c for the third one (fiber). For the first two, this results in a relation of type (6.2.9), for the third in the relation (6.2.10).

$$\frac{\sum_{r \in RAW} P_{rc} \cdot use_{rf}}{produce_f} \geq REQ_c \tag{6.2.9}$$

$$\frac{\sum_{r \in RAW} P_{rc} \cdot use_{rf}}{produce_f} \leq REQ_c \tag{6.2.10}$$

Both relations are non-linear since the variables $produce_f$ appear in the denominator, but as before it is possible to transform them into linear constraints by multiplying them by $produce_f$. Doing so, we obtain the constraints (6.2.3) and (6.2.4).

The constraints (6.2.5) guarantee that the quantities of raw material used remain within their limits of availability. The constraints (6.2.6) serve to satisfy the demand. And the last set of constraints (6.2.7) establishes that all variables are non-negative.

6.2.2 Code

```
#include <ilcplex/ilocplex.h>
#include <fstream>
#include"cmath"
#include"conio.h"

ILOSTLBEGIN
typedef IloArray<IloNumArray>        IloNumArray2base;
typedef IloArray<IloNumVarArray>     IloNumVarArray2;

IloInt i, j, k, RAW, COMP, FOOD;
IloNumArray Cost, Avail, REQ, DEM, PCG;
IloNumArray2 C;
IloNum PCB, PCGR, PCS;

//IloNumArray2 TC;


int main()

{

            ofstream fout("SolTP-.txt");

            // creating an environment
            IloEnv env;
            try {

                  //define_data(env);

                  Cost = IloNumArray(env);

                  Avail = IloNumArray(env);

                  DEM = IloNumArray(env);

                  REQ = IloNumArray(env);

                  PCG = IloNumArray(env);

                  C = IloNumArray2(env);

                  ifstream fin("data1.txt");

                  fin >> Cost >> Avail >> C >> DEM >> REQ >> PCG >> PCB >>
PCGR >> PCS;
```

```
if (!fin)

{
      cout << "Unable to open";
      exit(9);
}


RAW = Cost.getSize();

COMP = REQ.getSize();

FOOD = DEM.getSize();



double** xx;
xx = new double* [FOOD];
for (i = 0; i < FOOD; i++)
      xx[i] = new double[RAW];

double* yy;
yy = new double[FOOD];

IloModel model(env);


// variable declaration
/*----------------------------*/


IloNumVarArray2 x(env, FOOD);
for (i = 0; i < FOOD; i++)
{
      x[i] = IloNumVarArray(env, RAW, 0, IloInfinity,
ILOFLOAT);

}



IloNumVarArray y(env, FOOD, 0, IloInfinity, ILOFLOAT);


//variable declaration ends

/*OBJECTIVE FUNCTION*/
```

```
IloExpr T1(env);
for (i = 0; i < FOOD; i++)
{
      T1 += IloScalProd(Cost, x[i]);
}

IloExpr T2(env);
for (i = 0; i < FOOD; i++)
{
      T2 += IloScalProd(PCG, x[i]);
}

IloExpr T3(env);
for (i = 0; i < FOOD; i++)
{
      for (j = 0; j < RAW; j++)
            T3 += PCB * x[i][j];
}

IloExpr T4(env);
{
      for (j = 0; j < RAW; j++)
            T4 += PCGR * x[0][j];
}

IloExpr T5(env);
{
      for (j = 0; j < RAW; j++)
            T5 += PCS * x[1][j];
}


model.add(IloMinimize(env, T1 + T2 + T3 + T4 + T5));

//adding the constraints to the model


// Constraint 1
{
      for (i = 0; i < FOOD; i++)
            model.add(IloSum(x[i]) == y[i]);
}


//Constraint 2
```

```
for (i = 0; i < FOOD; i++)
{
      for (j = 0; j < 2; j++)

      {
            IloExpr DD(env);
            for (k = 0; k < RAW; k++)
            {
                  DD += C[j][k] * x[i][k];
            }
            model.add(DD >= REQ[j] * y[i]);
      }
}


//Constraint 3

for (i = 0; i < FOOD; i++)
{

      IloExpr DD1(env);
      for (k = 0; k < RAW; k++)
      {
            DD1 += C[2][k] * x[i][k];
      }
      model.add(DD1 <= REQ[2] * y[i]);              //
}



// Constraint 4
for (j = 0; j < RAW; j++)
{
      IloExpr DD2(env);
      for (i = 0; i < FOOD; i++)

      {
            DD2 += x[i][j];
      }
      model.add(DD2 <= Avail[j]);
}

// Constraint 5
for (i = 0; i < FOOD; i++)
{
      model.add(y[i] >= DEM[i]);
}
```

```
                  IloCplex cplex(model);
                  cplex.extract(model);
                  cplex.solve();

                  // outcome of the solver.
                  env.out() << "Solution Status is " << cplex.getStatus()
<< endl;
                  fout << "Solution Status is " << cplex.getStatus() <<
endl;
                  // the results
                  env.out() << " The objective value is " <<
cplex.getObjValue() << endl;
                  fout << " The objective value is " << "\t" <<
cplex.getObjValue() << endl;

                  cplex.exportModel("model.lp");



            }
            catch (IloException& ex)
            {
                  cerr << "Error: " << ex << endl;
            }
            catch (...)
            {
                  cerr << "Error" << endl;
            }
            // ending the program
            env.end();


      return 0;

}
```

6.2.3 Data

```
[0.13,0.17,0.12]
[11900,23500,750]
[[13.6,4.1,5],[7.1,2.4,0.3],[7,3.7,25]]
[9000,12000]
[9.5,2,6]
[0.25,0.25,0]
0.05
```

0.42
0.17